

TP FINAL

Diseño de Base de Datos I

Ingeniería Informática

Integrantes:

Nombre	Mail	Matrícula
Lucas Agüero	aguero.lucas@comunidad.ub.edu.ar	000-20-0985
Gianluca Boffa	gianluca.boffa@comunidad.ub.edu.ar	000-20-2057
Santiago Aparicio	santiago.aparicio@comunidad.ub.edu.ar	000-20-2335

Presentación:

“FutScore” es una web pensada para consultar resultados de partidos de fútbol masculino en vivo, sus tablas de posiciones y goleadores tanto de las ligas principales europeas como las sudamericanas.

Aquí la lista de ligas que se pueden revisar: **“UEFA Champions League”** (Europa); **“Primeira Liga”** (Portugal); **“Premier League”** (England); **“Eredivisie”** (Netherlands); **“Bundesliga”** (Alemania); **“Ligue 1”** (Francia); **“Serie A”** (Italia); **“La Liga”** (España); **“Championship”** (England); **“Serie A”** (Brazil); **“WorldCup 2026”** (Mundial); **“UEFA Eurocopa”** (Europa).

Además, la página también permite participar de un juego interactivo en el que el usuario debe predecir los pronósticos deportivos de cierto evento (Prode), compitiendo en un ranking global.

Herramientas Utilizadas:

Base de Datos / Backend: *Supabase (PostgreSQL)*, se eligió este debido a su robustez además de su soporte nativo para Triggers/Vistas y su sistema de autenticación integrado.

Frontend: *React + Vite*.

Estilizado: *Tailwind CSS* para la arquitectura de diseño con una estética moderna y fluida.

Consumo de Datos Externos: Integración con *“api.football-data.org”* para mantener actualizada la información deportiva.

Bitácora de Avance:

Semana 1: La primera semana consistió enteramente en pensar y llevar a cabo el diseño de la interfaz base de la página y su conexión con la API externa. Además de la refactorización visual (hecho en estilo “glass morphism”) y la adaptación responsiva.

Semana 2: La segunda semana nos enfocamos en el diseño del Modelo Relacional (DER) para integrar el juego interactivo de las predicciones (o Prode) y por último, la implementación de Base de Datos en *Supabase* (Tablas, Triggers, Vistas).

Semana 3: Finalmente, la última semana fue usada en su totalidad para la integración Full-Stack (Auth, envío de predicciones y visualización de ranking global).

Arquitectura de Base de Datos (PostgreSQL):

En este punto, el núcleo del proyecto gira en torno a cómo la base de datos gestiona el juego del Prode de manera autónoma, integrándose con una API externa a través del frontend.

Diagrama de Arquitectura del sistema:

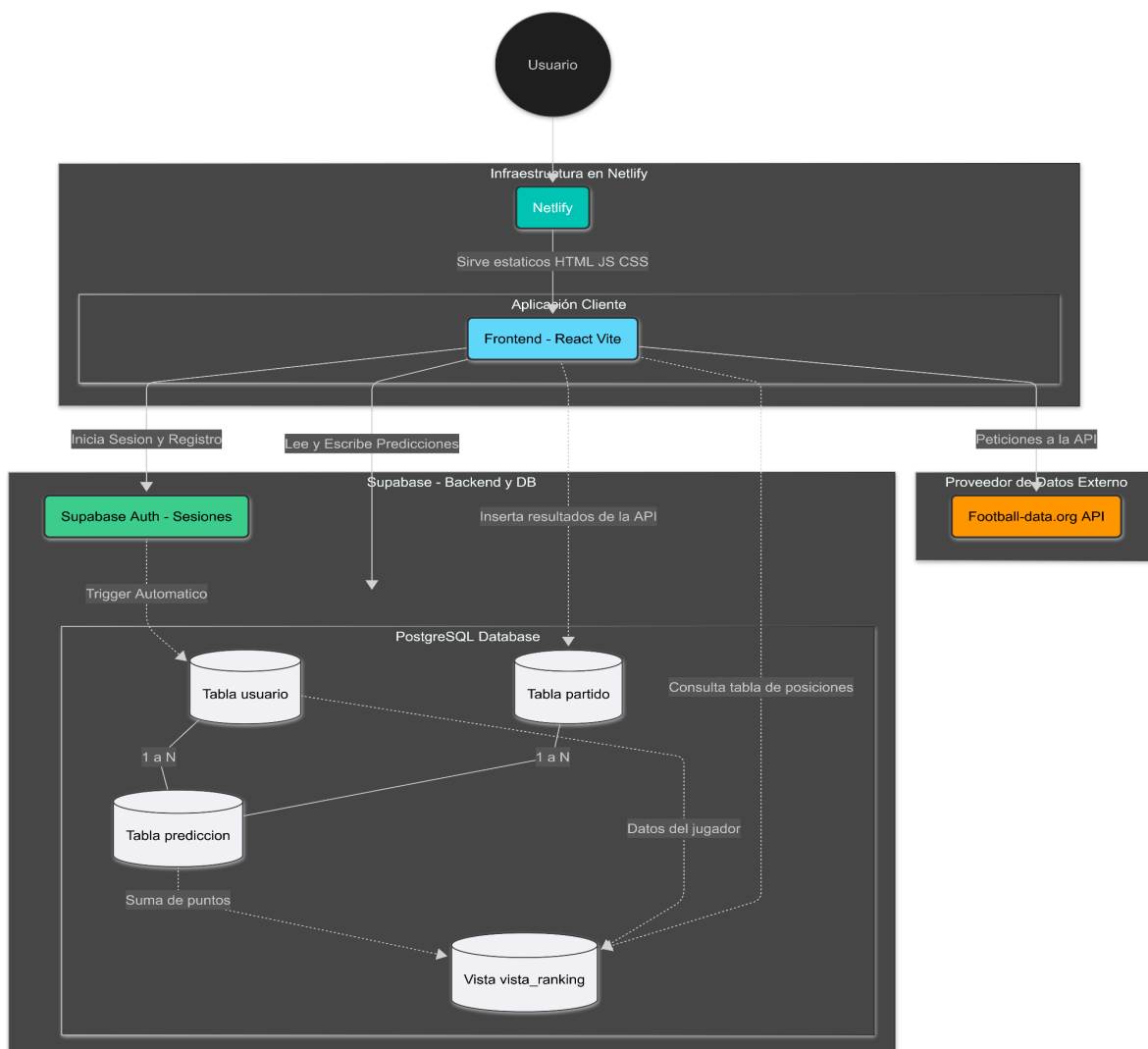
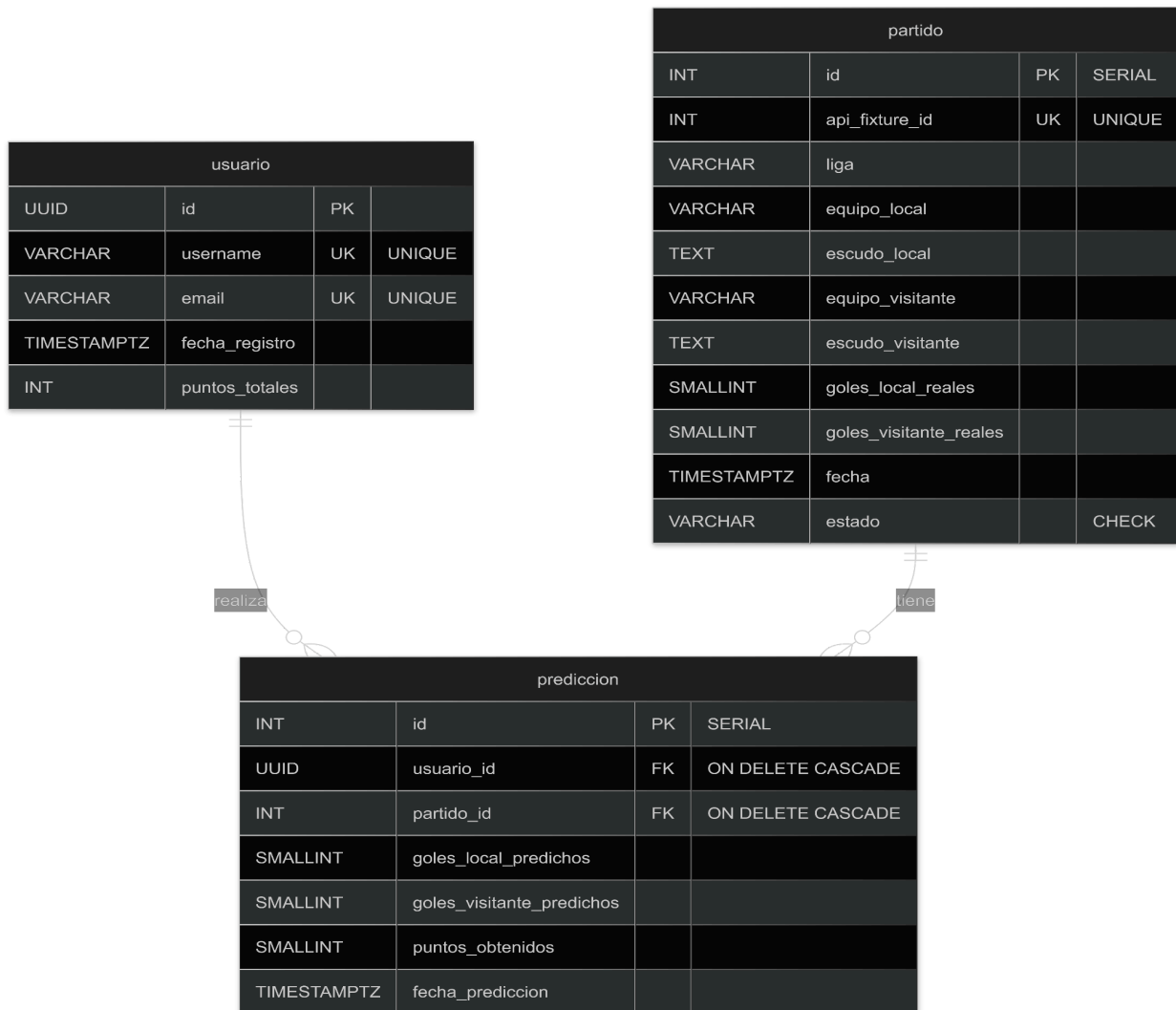


Diagrama de Entidad-Relación (DER):



Detalle de las Tablas y Restricciones (Constraints):

Usuario: Es la entidad principal vinculada a la autenticación.

- Los campos **username** y **email** tienen restricciones **UNIQUE** para evitar datos duplicados. El campo **id** es un **UUID** que funciona como Primary Key y se enlaza directamente con el sistema de autenticación.

Partido: Funciona como la "fuente de la verdad". Almacena qué equipos juegan, a qué liga pertenecen y, una vez finalizado, el resultado real.

- Los datos de los partidos se alimentan directamente de una API externa mediante un proceso de Upsert (Insert/Update) en *React*, utilizando **api_fixture_id** como clave única (**UNIQUE**).

- El campo `estado` posee un `CHECK CONSTRAINT` que solo permite los valores fijos: `'programado'`, `'en_curso'` o `'finalizado'`.

Predicción: Es una tabla intermedia transaccional que rompe la relación

Muchos-a-Muchos (N:M) entre `usuario` y `partido`.

- Posee claves foráneas `usuario_id` y `partido_id` con comportamiento `ON DELETE CASCADE`, o sea que si se borra un usuario o un partido en el futuro, sus predicciones desaparecen para mantener la integridad referencial.
- Tiene una restricción de tabla compuesta `UNIQUE` (`usuario_id`, `partido_id`) en la base de datos para garantizar matemáticamente que un usuario no pueda votar dos veces por un mismo partido, bloqueando intentos de trampa desde el servidor.

Código DDL (Data Definition Language)

```
-- 1. Tabla de Usuarios (Jugadores del Prode)
CREATE TABLE usuario (
    id                UUID PRIMARY KEY, -- Se vincula con auth.users.id
    username          VARCHAR(50) NOT NULL UNIQUE,
    email             VARCHAR(150) NOT NULL UNIQUE,
    fecha_registro    TIMESTAMPTZ DEFAULT NOW(),
    puntos_totales    INT DEFAULT 0
);

-- 2. Tabla de Partidos (Fuente de la verdad)
CREATE TABLE partido (
    id                SERIAL PRIMARY KEY,
    api_fixture_id    INT UNIQUE NOT NULL, -- ID que viene de la API de futbol
    liga              VARCHAR(100),
    equipo_local      VARCHAR(100) NOT NULL,
    escudo_local      TEXT,
    equipo_visitante  VARCHAR(100) NOT NULL,
    escudo_visitante  TEXT,
    goles_local_reales SMALLINT, -- Nulo hasta que termine el partido
    goles_visitante_reales SMALLINT,
    fecha             TIMESTAMPTZ NOT NULL,
    estado            VARCHAR(20) NOT NULL DEFAULT 'programado'
                    CHECK (estado IN ('programado', 'en_curso', 'finalizado'))
);

-- 3. Tabla de Predicciones (El Prode de los usuarios)
CREATE TABLE predicciones (
    id                SERIAL PRIMARY KEY,
    usuario_id        UUID NOT NULL REFERENCES usuario(id) ON DELETE CASCADE,
    partido_id        INT NOT NULL REFERENCES partido(id) ON DELETE CASCADE,
    goles_local_predichos SMALLINT NOT NULL,
```

```
goles_visitante_predichos SMALLINT NOT NULL,  
puntos_obtenidos          SMALLINT DEFAULT 0,  
fecha_prediccion          TIMESTAMPTZ DEFAULT NOW(),  
-- Restricción clave: Un usuario no puede predecir dos veces el mismo partido  
CONSTRAINT uq_usuario_partido UNIQUE (usuario_id, partido_id)  
);  
  
-- Índices para mejorar rendimiento de consultas  
CREATE INDEX idx_prediccion_usuario ON prediccion(usuario_id);  
CREATE INDEX idx_prediccion_partido ON prediccion(partido_id);  
CREATE INDEX idx_partido_estado     ON partido(estado);
```

Lógica de Negocio (Triggers y Vistas):

Para cumplir con los requisitos de la materia, la lógica de asignación de puntos NO ocurre en el código frontend, sino directamente en el motor de la base de datos:

Trigger (trg_actualizar_puntos): Cuando la tabla partido se actualiza y pasa a estado *'finalizado'*, este trigger dispara una función PL/pgSQL que cruza los goles reales con las predicciones. Se otorgan 3 puntos en caso de que el usuario haya acertado un resultado exacto, 1 punto en caso de acertar el ganador/empate, y 0 puntos por fallar. Automáticamente actualiza los puntos_totales del usuario.

Vista (vista_ranking): Una vista que consolida toda la información mediante un LEFT JOIN entre usuario y predicción. Para eso, cuenta la cantidad de participaciones, aciertos exactos y suma de puntos y deja que el Frontend solo tenga que hacer un SELECT * FROM vista_ranking para tener la tabla de posiciones lista.

Trigger de Auth (handle_new_user): Un trigger interno que crea automáticamente el perfil público en la tabla usuario cuando alguien se registra en el sistema de Auth de Supabase.

Documentación de instalación:

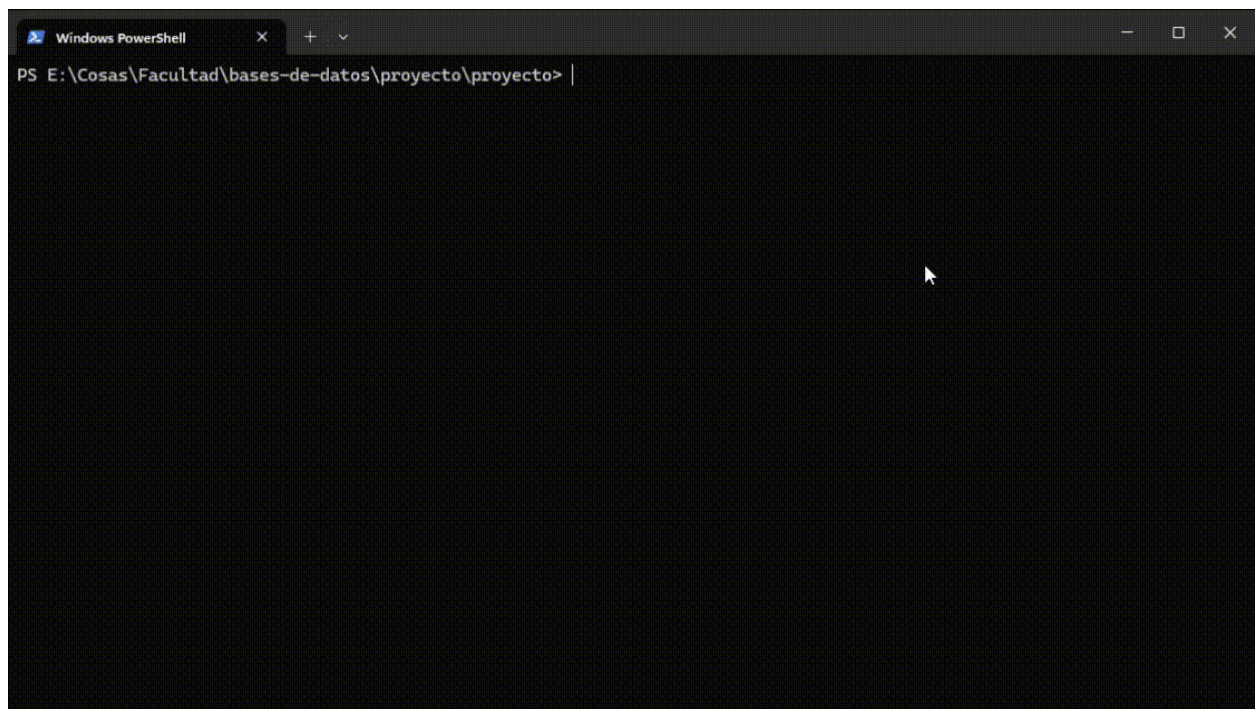
Para la instalación, hay unos requisitos previos que tener en cuenta:

1. Tener *Node.js* instalado
2. Tener una cuenta del tipo gratuita en *Supabase*
3. Tener una API Key del tipo gratuita de “*football-data.org*”

La instalación está conformada únicamente por dos pasos. En primer lugar, hace falta configurar el entorno local en un archivo `.env` en la raíz del proyecto con las siguientes claves: `VITE FOOTBALL DATA KEY=`, `VITE SUPABASE URL=` y `VITE SUPABASE ANON KEY=`.

Por último, solo quedaría ejecutar la aplicación que ya debería estar en funcionamiento con: `npm install` y `npm run dev`.

La aplicación estará disponible en **<http://localhost:5173>**.



Despliegue:

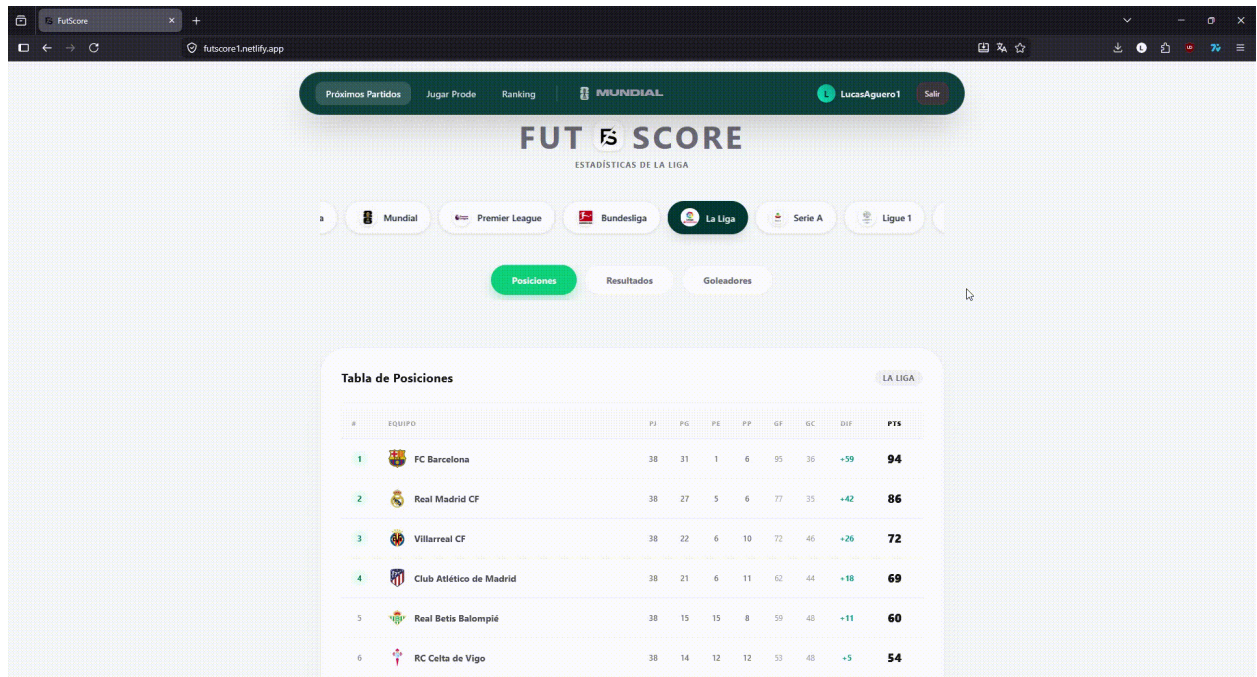
La aplicación fue desplegada en Netlify y se encuentra disponible en <https://futscore1.netlify.app/>.

Para su funcionamiento en producción se configuraron las variables de entorno

`VITE_FOOTBALL_DATA_KEY=`,

`VITE_SUPABASE_URL=`

`VITE_SUPABASE_ANON_KEY=` desde la sección de Environment Variables de Netlify.



The screenshot shows the FutScore application interface. At the top, there's a navigation bar with links for 'Próximos Partidos', 'Jugar Prode', 'Ranking', and 'MUNDIAL'. The main header displays 'FUT SCORE' and 'ESTADÍSTICAS DE LA LIGA'. Below this, there are tabs for different leagues: 'Mundial', 'Premier League', 'Bundesliga', 'La Liga' (selected), 'Serie A', and 'Ligue 1'. Under the 'La Liga' tab, there are three sub-tabs: 'Posiciones' (selected), 'Resultados', and 'Goleadores'. The main content area displays the 'Tabla de Posiciones' for 'LA LIGA'. The table lists the top 6 teams with their respective statistics.

#	EQUIPO	PJ	PG	PE	PP	GF	GC	DIF	PTS
1	FC Barcelona	38	31	1	6	95	36	+59	94
2	Real Madrid CF	38	27	5	6	77	35	+42	86
3	Villarreal CF	38	22	6	10	72	46	+26	72
4	Club Atlético de Madrid	38	21	6	11	62	44	+18	69
5	Real Betis Balompié	38	15	15	8	59	48	+11	60
6	RC Celta de Vigo	38	14	12	12	53	48	+5	54

Repositorio y Gestión de Código Fuente:

El proyecto fue gestionado mediante Git y alojado en GitHub, donde se registraron los avances de cada etapa de desarrollo. La utilización de un repositorio remoto permitió mantener trazabilidad sobre los cambios realizados, coordinar el trabajo en equipo y disponer de copias de seguridad del código.

[Repositorio](#)